

A4Q3

Lily Li

2025-07-31

Q3

(b)

```
df_law = read.csv("Lawnodes.csv")

lm_model <- lm(citecount ~ dissent, data = df_law)
summary(lm_model)

##
## Call:
## lm(formula = citecount ~ dissent, data = df_law)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -62.40 -41.72 -37.72 -13.40  925.60
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   41.719      1.375   30.332  <2e-16 ***
## dissent       20.678      2.401    8.611  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 113.1 on 10069 degrees of freedom
## Multiple R-squared:  0.007311,    Adjusted R-squared:  0.007212
## F-statistic: 74.15 on 1 and 10069 DF,  p-value: < 2.2e-16

alpha_hat <- coef(lm_model)[1]
beta_hat <- coef(lm_model)[2]

ATE <- beta_hat
lift <- beta_hat / alpha_hat

ATE

## dissent
## 20.67801
```

```
lift
```

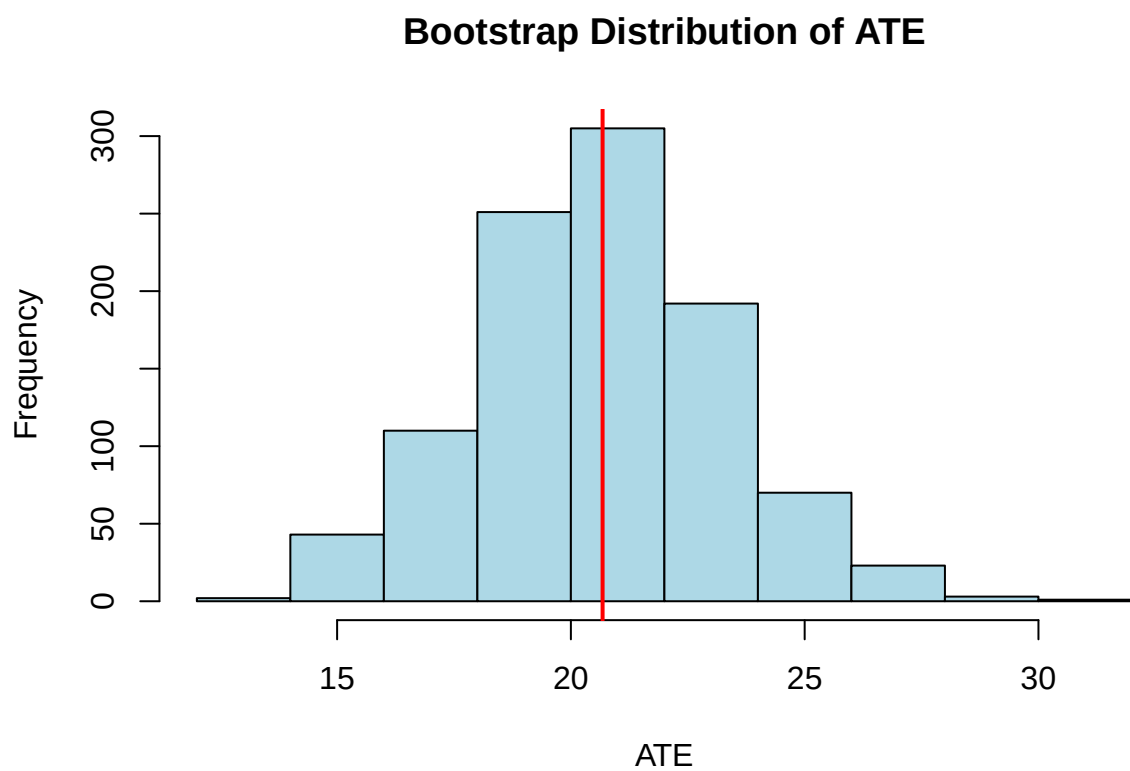
```
## dissent  
## 0.4956523
```

(c)

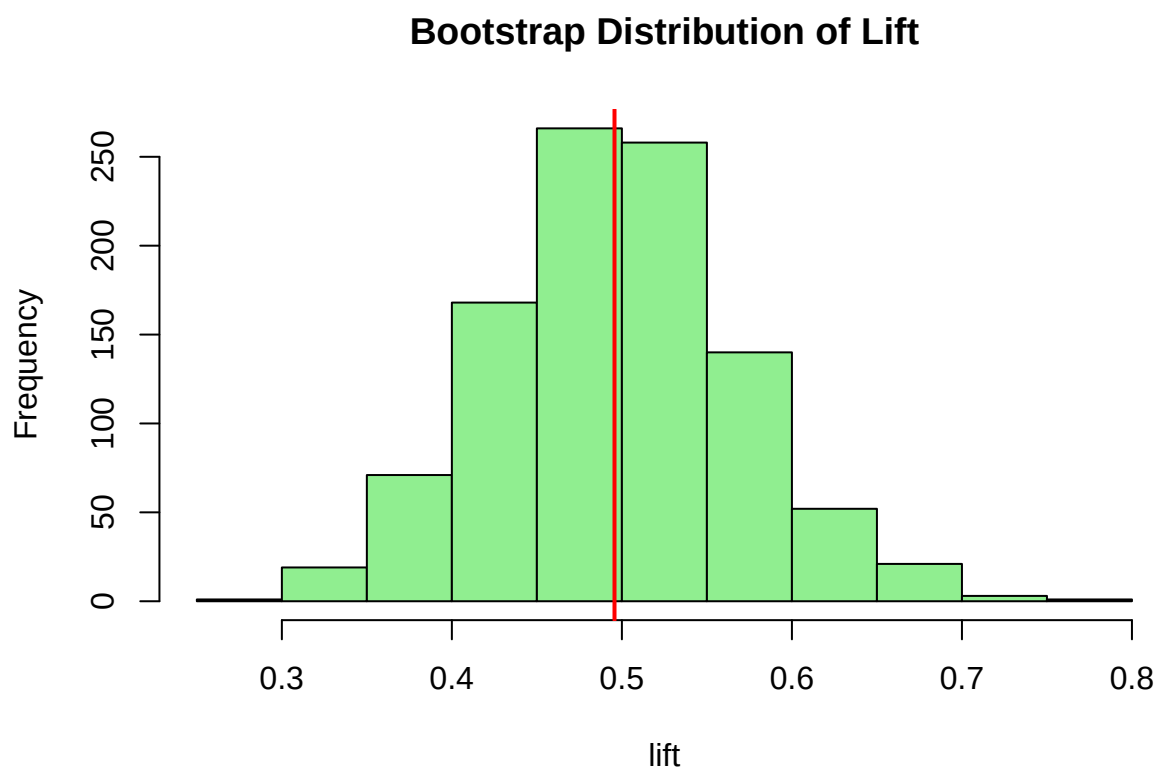
```
set.seed(123)  
B <- 1000  
n <- nrow(df_law)  
  
bootstrap_samples <- vector("list", B)  
  
for (b in 1:B) {  
  indices <- sample(1:n, size = n, replace=TRUE)  
  bootstrap_samples[[b]] <- df_law[indices, ]  
}
```

(d)

```
ATE_star <- numeric(B)  
lift_star <- numeric(B)  
  
for (b in 1:B) {  
  model_b <- lm(citecount ~ dissent, data = bootstrap_samples[[b]])  
  alpha_b <- coef(model_b)[1]  
  beta_b <- coef(model_b)[2]  
  ATE_star[b] <- beta_b  
  lift_star[b] <- beta_b / alpha_b  
}  
  
hist(ATE_star, main = "Bootstrap Distribution of ATE", xlab = "ATE", col = "lightblue")  
  
abline(v = ATE, col = "red", lwd = 2)
```



```
hist(lift_star, main = "Bootstrap Distribution of Lift", xlab = "lift", col = "lightgreen")  
abline(v = lift, col = "red", lwd = 2)
```



(e)

```
ATE_se <- sd(ATE_star)
lift_se <- sd(lift_star)

ATE_CI <- c(ATE - 1.96 * ATE_se, ATE + 1.96 * ATE_se)
lift_CI <- c(lift - 1.96 * lift_se, lift + 1.96 * lift_se)

print(ATE_CI)
```

```
## dissent dissent
## 15.45604 25.89998
```

```
print(lift_CI)
```

```
## dissent dissent
## 0.3541126 0.6371919
```

(f)

```
ATE_CI_percentile <- quantile(ATE_star, probs = c(0.025, 0.975))
lift_CI_percentile <- quantile(lift_star, probs = c(0.025, 0.975))
```

```
print(ATE_CI_percentile)
```

```
##      2.5%      97.5%
## 15.40942 26.03704
```

```
print(lift_CI_percentile)
```

```
##      2.5%      97.5%
## 0.3582020 0.6496791
```

(g)

```
bootstrap_t_interval_new <- function(S, a, confidence, B, D) {

  Pstar <- S
  aPstar <- a(S) # Attribute value on original sample
  n <- nrow(S)

  # Outer bootstrap: compute a(S*) and its z-score
  bvals <- sapply(1:B, FUN = function(b) {
    indices <- sample(1:n, size = n, replace = TRUE)
    Sstar <- Pstar[indices,]
    aSstar <- a(Sstar)

    # Inner bootstrap: estimate SD of a(S*)
    Pstarstar <- Sstar
    inner_n <- nrow(Sstar)
    SD_aSstar <- sd(sapply(1:D, FUN = function(d) {

      inner_indices <- sample(1:inner_n, size = inner_n, replace = TRUE)
      Sstarstar <- Sstar[inner_indices,]
      a(Sstarstar)
    })))

    z <- (aSstar - aPstar) / SD_aSstar
    c(aSstar, z)
  })

  # Extract values
  SDhat <- sd(bvals[1, ]) # Standard deviation of aSstar values
  zVals <- bvals[2, ]    # z-scores
}
```

```

# Quantile cutoffs
alpha <- 1 - confidence
cValues <- quantile(zVals, probs = c((1 - confidence) / 2, (1 + confidence) / 2), na.rm = TRUE)
cLower <- min(cValues)
cUpper <- max(cValues)

# Compute interval
interval <- c(
  lower = aPstar - cUpper * SDhat,
  middle = aPstar,
  upper = aPstar - cLower * SDhat
)

return(interval)
}

```

```

ATE_func <- function(data) {
  model <- lm(citecount ~ dissent, data = data)
  coef(model)[2]
}

lift_func <- function(data) {
  model <- lm(citecount ~ dissent, data = data)
  beta <- coef(model)[2]
  alpha <- coef(model)[1]
  beta / alpha
}

B_small <- 100
D <- 100

ATE_boot_t_CI <- bootstrap_t_interval_new(df_low, ATE_func, confidence = 0.95, B = B_small, D = D)

lift_boot_t_CI <- bootstrap_t_interval_new(df_low, lift_func, confidence = 0.95, B = B_small, D = D)

print(ATE_boot_t_CI)

```

```

## lower.dissent middle.dissent upper.dissent
##      13.97330      20.67801      27.21160

```

```

print(lift_boot_t_CI)

```

```

## lower.dissent middle.dissent upper.dissent
##      0.4150516      0.4956523      0.6092502

```

(h)

This approach is simple and computationally efficient, as it only requires estimating the standard error and assuming normality to construct the confidence interval. However, it heavily relies on the assumption that the sampling distribution of the estimator is approximately normal. When the true distribution is skewed or the sample size is small, this assumption may not hold, leading to inaccurate or misleading confidence

intervals. The method does not adapt to the actual shape of the distribution, so it can underestimate or overestimate the tails.

The percentile method is intuitive and straightforward to implement, as it constructs confidence intervals directly from the empirical quantiles of the bootstrap distribution. It doesn't rely on any parametric assumptions, which is an advantage. However, it can still produce biased intervals when the bootstrap distribution is skewed or asymmetric because it does not correct for the bias or variability in the estimator. This method assumes that the bootstrap distribution mimics the sampling distribution well, which is not always the case, especially with small samples or skewed data.

This method adjusts for both bias and skewness by using a studentized statistic (i.e., the estimator divided by its standard error). As a result, it generally produces more accurate and reliable confidence intervals, especially in the presence of non-normality or heterogeneous variability. By standardizing the estimator in each resample and using a double bootstrap to estimate variability, it accounts for the uncertainty more robustly. However, this increased accuracy comes at a computational cost—since it involves nested resampling, the method is significantly more time-consuming and resource-intensive.